Do people prefer shorter or longer video on TikTok?

-Python Project-

Table of Contents

Disclaimer: 2

1. Introduction 2

2. Data Exploration 2

2.1 Statistical summaries 2

2.2 Categorical columns 4

2.3 Numerical columns 6

2.4. Outliers check 12

2.5. Duplication check 14

2.6 Pattern 14

3. Data Preparation 15

3.1. Missing values 15

3.2. Irrelevant columns 16

3.3. Creating additional variables 17

3.4 Why using median? 17

4. Analysis 18

Does video duration affect engagement? 21

5. Conclusion 22

Disclaimer: *The dataset used in this project contains video duration within the range of 5 to 60 seconds. While this range does not include longer video duration that also exist on TikTok, the limitation is inherent to the dataset and not a result of data filtering performed by the researcher. Importantly, this duration range is consistent with the typical length of short-form videos on TikTok, particularly during the period when short videos (under 60 seconds) were the platform's most widely used format.*

Therefore, although the findings of this project primarily reflect engagement behaviour for short-form content, they remain valid and representative of many user interactions on TikTok. The conclusions should not be generalised to longer-format TikTok videos, but they are appropriate for analysing.

Good

1. Introduction

In recent years, short-form video platforms have shown their rapid growth and changed how people consume online content. As one of the leading platforms with its short and fast paced videos, TikTok has been influencing trends and user engagement globally. It offers a large variety of content with different duration, typically ranges from 5 seconds to 1 minute on average. Moreover, the popularity or the engagement of a video can be measured through its number of likes, shares, comments and views it received.

The aim of this report is to explore the impact of video duration on user engagement on TikTok. This report also aims to seek if shorter or longer videos lead to a higher engagement and the relationship between video duration and those engagement metrics.

The dataset is available at TikTok User Engagement Data.

Good intro.

2. Data Exploration

We will need to import libraries like Pandas, Matplotlib and Seaborn for data processing and visualisation. Then we will import the TikTok dataset to begin processing them.

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv('/Volumes/project/dataset/tiktok_dataset/tiktok_dataset.csv')
```

2.1 Statistical summaries

After loading the dataset, data.shape() will be used to get an overview of the whole dataset.

```
data.shape
```

```
(19382, 12)
```

This dataset has 19382 rows and 12 columns.

Let's view the first 5 rows of the data frame.

```
data.head()
```

	#	claim_status	video_id	video_duration_sec	video_transcription_text	verified_status	author_ban_status	video_view_count	video_lil
0	1	claim	7017666017	59	someone shared with me that drone deliveries a...	not verified	under review	343296.0	
1	2	claim	4014381136	32	someone shared with me that there are more mic...	not verified	active	140877.0	
2	3	claim	9859838091	31	someone shared with me that american industria...	not verified	active	902185.0	
3	4	claim	1866847991	25	someone shared with me that the metro of st. p...	not verified	active	437506.0	
4	5	claim	7105231098	19	someone shared with me that the number of busi...	not verified	active	56167.0	

```
data.head()
```

:t	verified_status	author_ban_status	video_view_count	video_like_count	video_share_count	video_download_count	video_comment_count
e ..	not verified	under review	343296.0	19425.0	241.0	1.0	0.0
e ..	not verified	active	140877.0	77355.0	19034.0	1161.0	684.0
e ..	not verified	active	902185.0	97690.0	2858.0	833.0	329.0
e ..	not verified	active	437506.0	239954.0	34812.0	1234.0	584.0
e ..	not verified	active	56167.0	34987.0	4110.0	547.0	152.0

Overall, this dataset is well-structured with a clean and consistent formatting. We will use data.info() to have a closer look of the whole dataset.

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19382 entries, 0 to 19381
Data columns (total 12 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   #                         19382 non-null  int64
 1   claim_status              19084 non-null  object
 2   video_id                  19382 non-null  int64
 3   video_duration_sec        19382 non-null  int64
 4   video_transcription_text  19084 non-null  object
 5   verified_status           19382 non-null  object
 6   author_ban_status         19382 non-null  object
 7   video_view_count          19084 non-null  float64
 8   video_like_count          19084 non-null  float64
 9   video_share_count         19084 non-null  float64
 10  video_download_count      19084 non-null  float64
 11  video_comment_count       19084 non-null  float64
dtypes: float64(5), int64(3), object(4)
memory usage: 1.8+ MB
```

The data frame consists of 12 columns: 4 text columns (claim_status, video_transcription_text, verified_status, author_ban_status), and 8 numeric columns (video_id and mainly video's metrics like duration or views, likes…)

And there are 7 columns that have missing values: claim_status, video_transcription_text, video_like_count, video_view_count, video_share_count, video_download_count, video_comment_count.

Good, should also mention the percentage wise to see how significant it is.

```
missing_data = data.isnull().sum()

missing_val = missing_data[missing_data > 0]
missing_val
```

```
claim_status              298
video_transcription_text  298
video_view_count          298
video_like_count          298
video_share_count         298
video_download_count      298
video_comment_count       298
dtype: int64
```

2.2 Categorical columns

For a better processing purpose, we will put them into 2 groups: categorical and numerical.

```
categorical = []
for column in data.columns:
    if data[column].dtypes == 'O':
        categorical.append(column)

numerical = []
for column in data.columns:
    if data[column].dtypes != 'O':
        numerical.append(column)

cat_cols = data[categorical]
num_cols = data[numerical]
```

```
print("Categorical columns:\n", categorical)
print("\nNumerical columns:\n", numerical)
```

```
Categorical columns:
 ['claim_status', 'video_transcription_text', 'verified_status', 'author_ban_status']

Numerical columns:
 ['#', 'video_id', 'video_duration_sec', 'video_view_count', 'video_like_count', 'video_share_count', 'video_download_coun
t', 'video_comment_count']
```

Now that the columns are split into 2 groups, we will have a look at its data statistic:

```
print("Categorical desciption:\n")
cat_cols.describe()
```

Categorical desciption:

	claim_status	video_transcription_text	verified_status	author_ban_status
count	19084	19084	19382	19382
unique	2	19012	2	3
top	claim	a friend read in the media a claim that badmi...	not verified	active
freq	9608	2	18142	15663

- **Claim_status:** 19084 non-missing values, with 2 statuses ‘claim’ and ‘non-claim’. The ‘claim’ status being the most common category which appears 9 608 times in the dataset => Half of all videos include a claim.
- **Video_transcription_text:** 19084 non-missing values, with 19 012 different transcription texts, and the top one appears 2 times => Each video contains different audio content.
- **Verified_status:** 19382 values (no missing value), with 2 statuses ‘verified’ and ‘non-verified’. The ‘non-verified’ status is dominating with 18142 rows => Matches TikTok demographics as most of the creators/users are not verified.
- **Author_ban_status:** no missing value, with 3 statuses being ‘active’, ‘banned’ and ‘under review’ => The majority of accounts are active, suggesting the dataset contains mostly end-user content. ?? this doesn't make sense

Should explain what claim status is in business context.

This is interesting because usually you would expect video transcription to be unique but here it appears twice → Are they the same video ? Duplicated ? AI generated ?

2.3 Numerical columns

```
print("Numerical desciption:\n")
num_cols.describe()
```

```
Numerical desciption:
```

	#	video_id	video_duration_sec	video_view_count	video_like_count	video_share_count	video_download_count
count	19382.000000	1.938200e+04	19382.000000	19084.000000	19084.000000	19084.000000	19084.000000
mean	9691.500000	5.627454e+09	32.421732	254708.558688	84304.636030	16735.248323	1049.429627
std	5595.245794	2.536440e+09	16.229967	322893.280814	133420.546814	32036.174350	2004.299894
min	1.000000	1.234959e+09	5.000000	20.000000	0.000000	0.000000	0.000000
25%	4846.250000	3.430417e+09	18.000000	4942.500000	810.750000	115.000000	7.000000
50%	9691.500000	5.618664e+09	32.000000	9954.500000	3403.500000	717.000000	46.000000
75%	14536.750000	7.843960e+09	47.000000	504327.000000	125020.000000	18222.000000	1156.250000
max	19382.000000	9.999873e+09	60.000000	999817.000000	657830.000000	256130.000000	14994.000000

```
print("Numerical desciption:\n")
num_cols.describe()
```

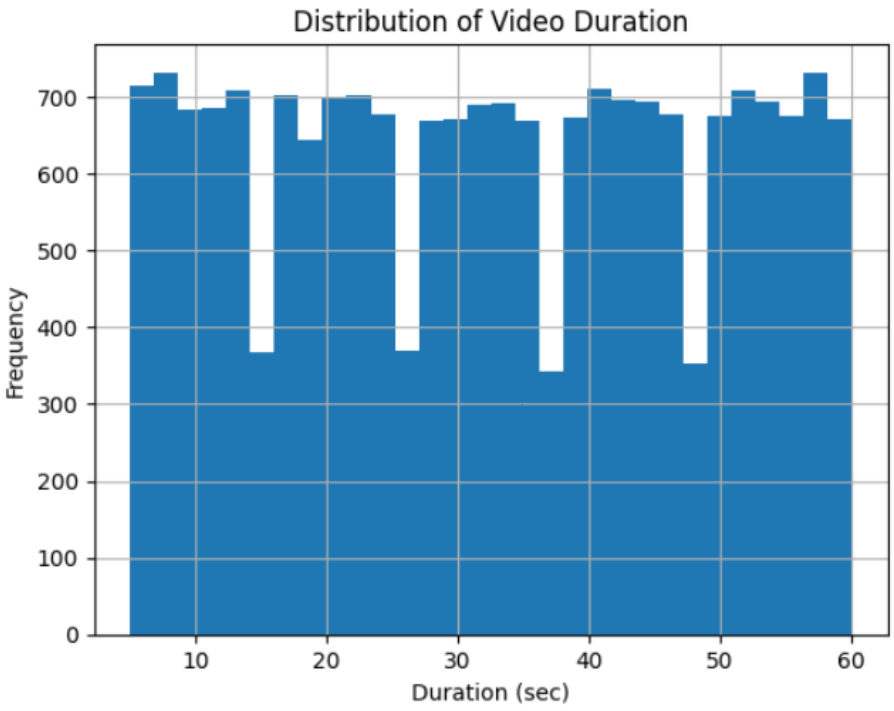
```
Numerical desciption:
```

#	video_id	video_duration_sec	video_view_count	video_like_count	video_share_count	video_download_count	video_comment_count
0	1.938200e+04	19382.000000	19084.000000	19084.000000	19084.000000	19084.000000	19084.000000
0	5.627454e+09	32.421732	254708.558688	84304.636030	16735.248323	1049.429627	349.312146
4	2.536440e+09	16.229967	322893.280814	133420.546814	32036.174350	2004.299894	799.638865
0	1.234959e+09	5.000000	20.000000	0.000000	0.000000	0.000000	0.000000
0	3.430417e+09	18.000000	4942.500000	810.750000	115.000000	7.000000	1.000000
0	5.618664e+09	32.000000	9954.500000	3403.500000	717.000000	46.000000	9.000000
0	7.843960e+09	47.000000	504327.000000	125020.000000	18222.000000	1156.250000	292.000000
0	9.999873e+09	60.000000	999817.000000	657830.000000	256130.000000	14994.000000	9599.000000

- **Video_duration_sec:**

- Most videos range from 5 to 60 seconds, with a median of 32 seconds and the mean also reaches approximately 32 seconds.
- A histogram will be produced to further investigate its distribution.

```
data["video_duration_sec"].hist(bins=30)
plt.title("Distribution of Video Duration")
plt.xlabel("Duration (sec)")
plt.ylabel("Frequency")
```

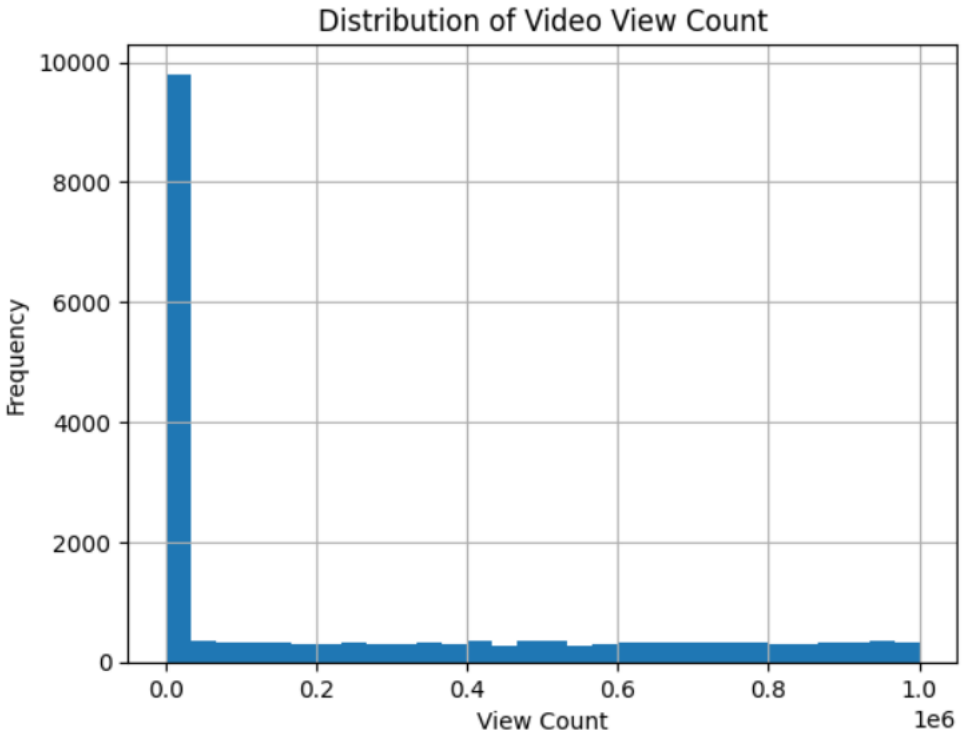


=> The duration variable seems to be evenly distributed.

- **Video_view_count:**

- The minimum view being 20 indicates that some videos barely reached audiences, while some videos went viral as the maximum reached approximately 1 million.

```
data['video_view_count'].hist(bins=30)
plt.title("Distribution of Video View Count")
plt.xlabel("View Count")
plt.ylabel("Frequency")
```



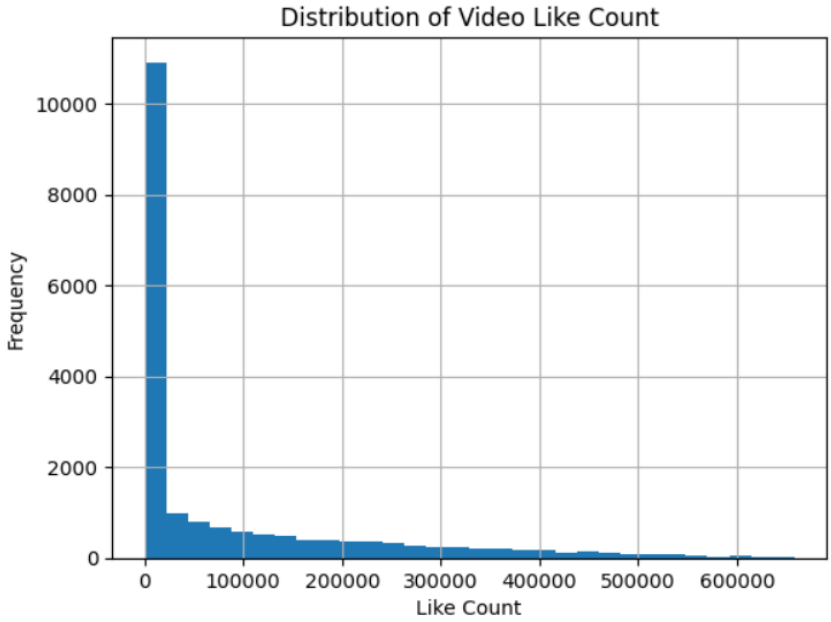
=> The distribution is right skewed while the mean (254708.55) is way larger than the median (9954.5) as shown in the statistical table above.

→ Explain this in business language, what does it mean to be right-skewed in this case?

- **Video_like_count:**

- Some videos were viewed but didn't receive any like, the jump from median (3403.5) to mean (84304.636) indicates right skewness as some videos received a very high number of likes.

```
data['video_like_count'].hist(bins=30)
plt.title("Distribution of Video Like Count")
plt.xlabel("Like Count")
plt.ylabel("Frequency")
```

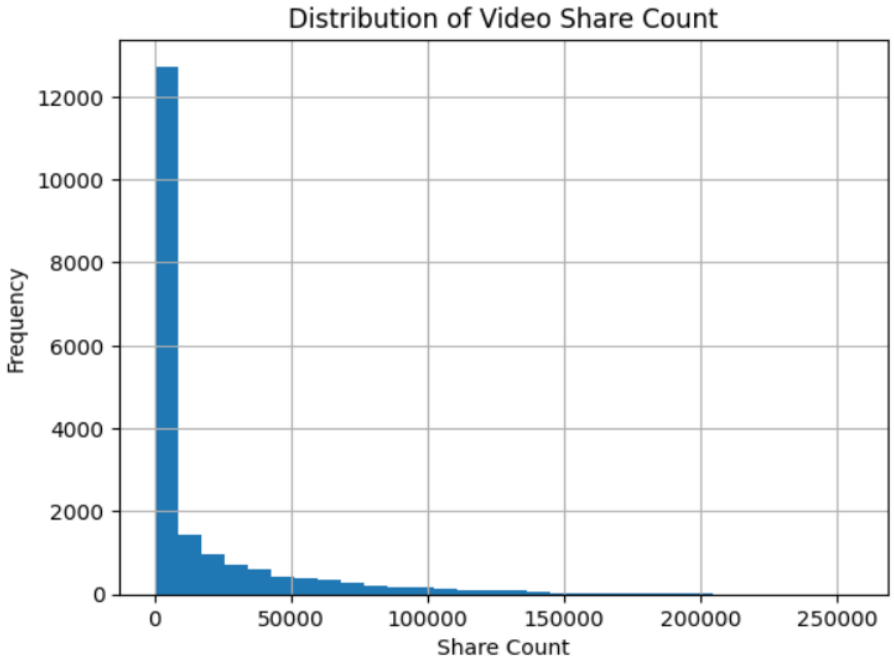


=> The histogram shows a large number of videos receiving relatively low like counts, as indicated by the clear spike near zero. In contrast, a tiny number of videos received remarkable high numbers of likes, forming a long right tail.

- **Video_share_count:**

- Some videos received very little to almost no share, while some received very high share amount.

```
data['video_share_count'].hist(bins=30)
plt.title("Distribution of Video Share Count")
plt.xlabel("Share Count")
plt.ylabel("Frequency")
```

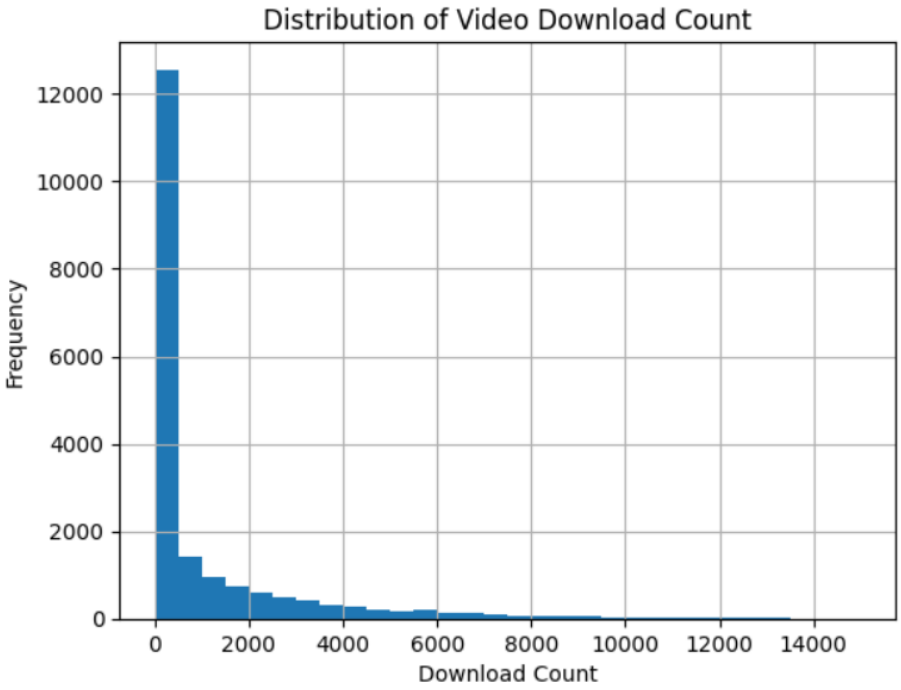


=> Again, the distribution is consistent with earlier metrics as the graph shows that most videos receive relatively low numbers of shares, indicated by the spike near zero. And a small portion of the distribution with extremely high share counts, resulting in a long right tail.

- **Video_download_count:**

- Downloads are much lower than likes or shares, the majority received very few downloads.

```
data['video_download_count'].hist(bins=30)
plt.title("Distribution of Video Download Count")
plt.xlabel("Download Count")
plt.ylabel("Frequency")
```

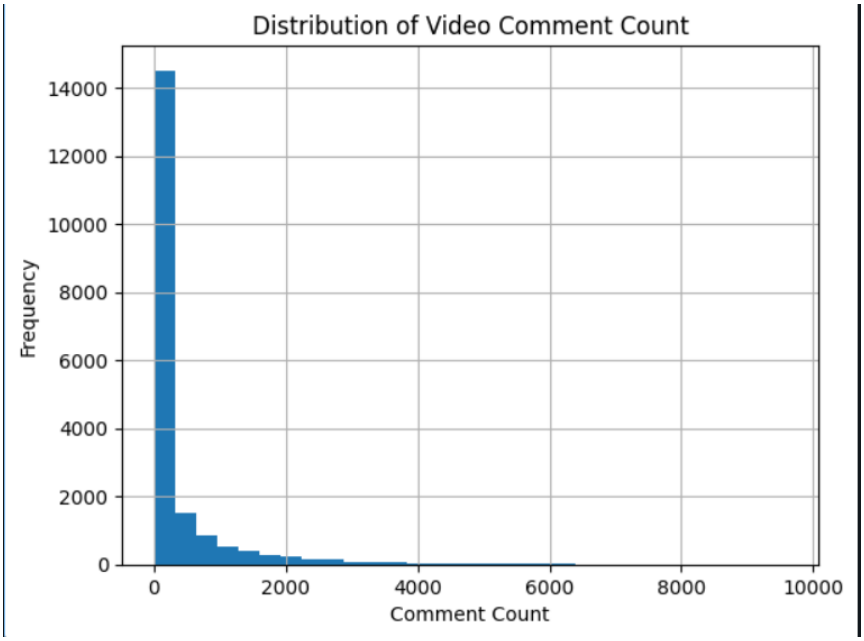


=> Following the same pattern observed in views, likes, and shares, the distribution is also heavily right-skewed.

- **Video_comment_count:**

- Comment counts seem to be the most skewed of all metrics as many videos received less than 10 comments with a small number of videos with hundreds to thousands of comments.

```
data['video_comment_count'].hist(bins=30)
plt.title("Distribution of Video Comment Count")
plt.xlabel("Comment Count")
plt.ylabel("Frequency")
```



In conclusion, across all engagement metrics, their distributions show a very consistent right-skewed pattern. Highlighted the uneven in engagement, making the distribution tails stretch far to the right.

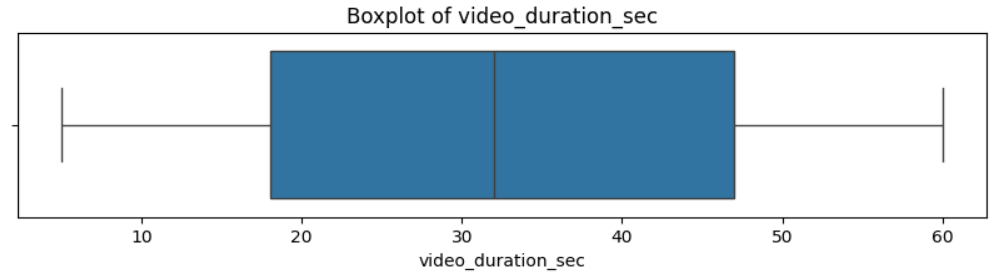
Good

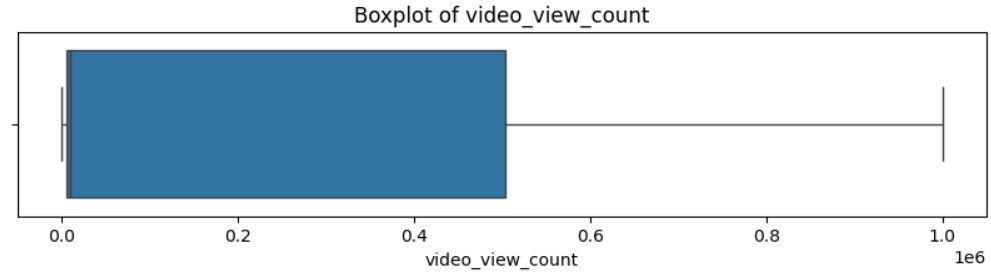
2.4. Outliers check

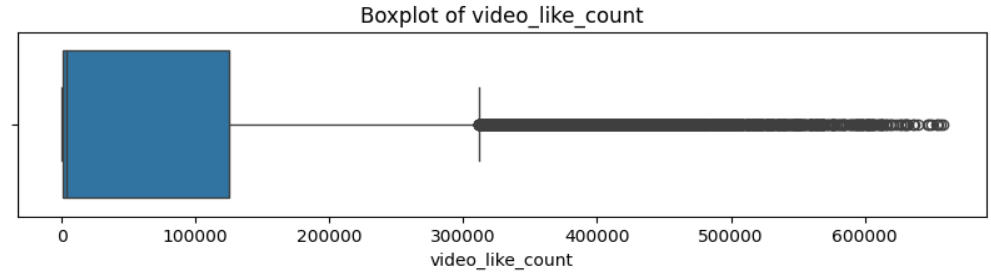
Multiple box plots will be produced to check for outliers:

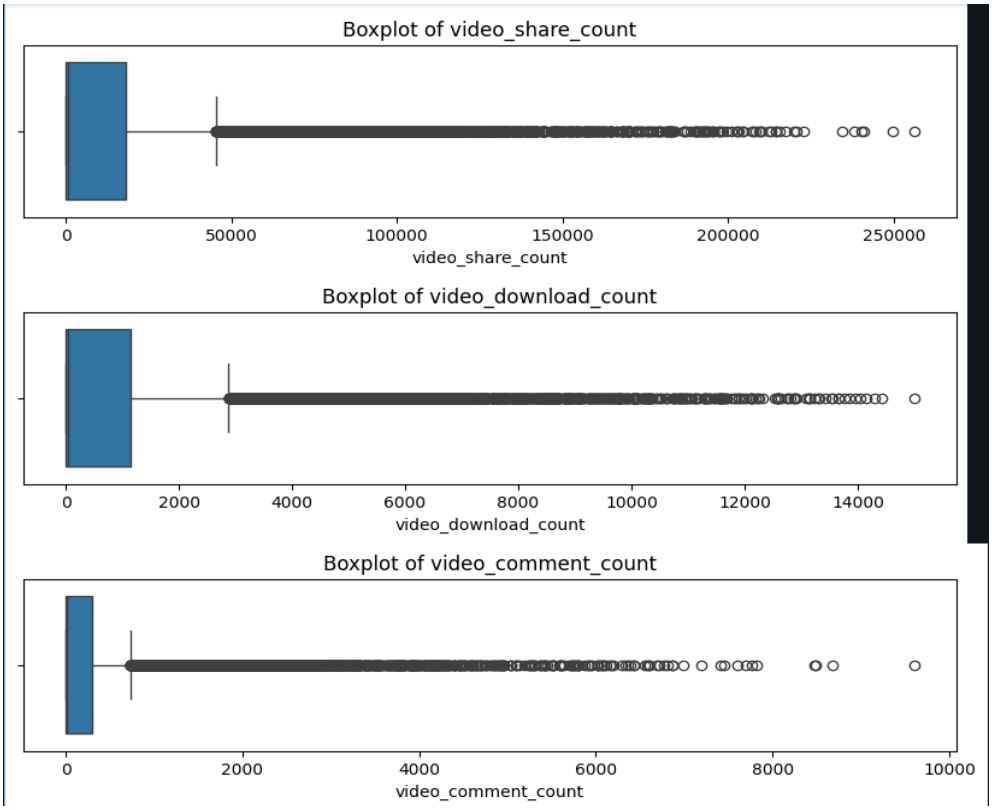
```
for col in ['video_duration_sec','video_view_count','video_like_count','video_share_count',
            'video_download_count','video_comment_count']:

    plt.figure(figsize=(10,2))
    sns.boxplot(x=data[col])
    plt.title(f"Boxplot of {col}")
```


Boxplot of video_duration_sec
10
20
30
40
50
60
video_duration_sec


Boxplot of video_view_count
0.0
0.2
0.4
0.6
0.8
1.0
1e6
video_view_count


Boxplot of video_like_count
0
100000
200000
300000
400000
500000
600000
video_like_count


Boxplot of video_share_count
0
50000
100000
150000
200000
250000
video_share_count
Boxplot of video_download_count
0
2000
4000
6000
8000
10000
12000
14000
video_download_count
Boxplot of video_comment_count
0
2000
4000
6000
8000
10000
video_comment_count

- All engagement metrics show a skewed distribution with extremely long upper whiskers and a large number of outliers.
- On the other hand, video duration shows a much more balanced distribution with no extreme outliers with most videos concentrate between 20–40 seconds.

2.5. Duplication check

A duplication check will now be performed to find out if there is any duplicated entry:

```
print("Duplicated rows: ", data.duplicated().sum())
print("Duplicated categorical rows: ", cat_cols.duplicated().sum())
print("Duplicated numerical rows: ", num_cols.duplicated().sum())
```

```
Duplicated rows:  0
Duplicated categorical rows:  330
Duplicated numerical rows:  0
```

- No fully duplicated rows as every video entry is unique.
- It is expected even when some categorical values appear more than once, such as many users are 'not verified' or many videos are marked as 'claim', and this does not represent duplicated data.
- Engagement metrics are distinct as numerical columns showed no duplication.

Good

2.6 Pattern

Now we will check for any special pattern of the video duration and view count using a scatterplot:

```
plt.figure(figsize=(12,8))
plt.scatter(data["video_duration_sec"], data["video_view_count"])
plt.title("Video Duration vs View Count")
plt.xlabel("Duration (seconds)")
plt.ylabel("View Count")
```



= > This plot is overplotting and every dot is overlapping due to the large sample size, making it unreadable. The same is expected for the other remain metrics. But from this plot above, we can conclude that it shows no clear relationship between video duration and view count, indicating that duration alone is not a strong predictor of video performance. Therefore, to check their relationship more carefully, we will need to group durations into different intervals which will be covered in section 3.3.

3. Data Preparation

3.1. Missing values

As mentioned on section 2, there are 298 missing values in 7 columns. These rows lacked all important engagement metrics such as views, likes, shares… Imputing those missing values will create fake data and twist the transparency of the analysis. So, it is better to remove those values than imputing them to ensure the reliability of this analysis.

Since these entries all have missing values in the same rows, dropna() will be used to remove them:

Reasonable approach

```
#Dropping missing value rows
remove_missing = data.dropna(inplace = True)

data.isnull().sum()
```

```
#                         0
claim_status              0
video_id                  0
video_duration_sec        0
video_transcription_text  0
verified_status           0
author_ban_status         0
video_view_count          0
video_like_count          0
video_share_count         0
video_download_count      0
video_comment_count       0
dtype: int64
```

All missing rows are removed, but just to make sure, we can double check again by counting the null values in each column => No more null values!!

3.2. Irrelevant columns

As the main objective of this analysis is to discover if shorter or longer videos are more popular. Other variables like 'claim_status', 'video_transcription_text', 'verified_status'and 'author_ban_status' are irrelevant and will also be removed because it is not required for this analysis purpose.

```
data.drop(['claim_status','video_transcription_text','author_ban_status','verified_status'], axis=1, inplace=True)
data
```

	#	video_id	video_duration_sec	video_view_count	video_like_count	video_share_count	video_download_count	video_comm
0	1	7017666017	59	343296.0	19425.0	241.0	1.0	
1	2	4014381136	32	140877.0	77355.0	19034.0	1161.0	
2	3	9859838091	31	902185.0	97690.0	2858.0	833.0	
3	4	1866847991	25	437506.0	239954.0	34812.0	1234.0	
4	5	7105231098	19	56167.0	34987.0	4110.0	547.0	
...	...	...	...	...	...	...	...	
19079	19080	1492320297	49	6067.0	423.0	81.0	8.0	
19080	19081	9841347807	23	2973.0	820.0	70.0	3.0	
19081	19082	8024379946	50	734.0	102.0	7.0	2.0	
19082	19083	7425795014	8	3394.0	655.0	123.0	11.0	
19083	19084	4094655375	58	5034.0	815.0	281.0	11.0	

19084 rows × 8 columns

=> All irrelevant columns removed

3.3. Creating additional variables

To reduce noise from overplotting in section 2 and to analyse engagement trends across video length, a new variable 'duration_group' was created by separating video durations into four bins:

Very good!

- Bin 1/Group 1: 5-15 seconds
- Bin 2/Group 2: 15-30 seconds
- Bin 3/Group 3: 30-45 seconds
- Bin 4/Group 4: 45-60 seconds

```
data["duration_group"] = pd.cut(
    data["video_duration_sec"],
    bins=[5, 15, 30, 45, 60],
    labels=["5–15s", "15–30s", "30–45s", "45–60s"]
)

data.head()
```

ation_sec	video_transcription_text	verified_status	author_ban_status	video_view_count	video_like_count	video_share_count	video_download_count	video_comment_count	duration_group
59	someone shared with me that drone deliveries a...	not verified	under review	343296.0	19425.0	241.0	1.0	0.0	45–60s
32	someone shared with me that there are more mic...	not verified	active	140877.0	77355.0	19034.0	1161.0	684.0	30–45s
31	someone shared with me that american industria...	not verified	active	902185.0	97690.0	2858.0	833.0	329.0	30–45s
25	someone shared with me that the metro of st. p...	not verified	active	437506.0	239954.0	34812.0	1234.0	584.0	15–30s
19	someone shared with me that the number of busi...	not verified	active	56167.0	34987.0	4110.0	547.0	152.0	15–30s

3.4 Why using median?

Median values were used instead of means because the engagement metrics are right-skewed, with a small number of viral videos having significant high counts. Making the mean increase and inaccurate to represent a video performance. Using the median will provide a more accurate measure of trends for these skewed distributions.

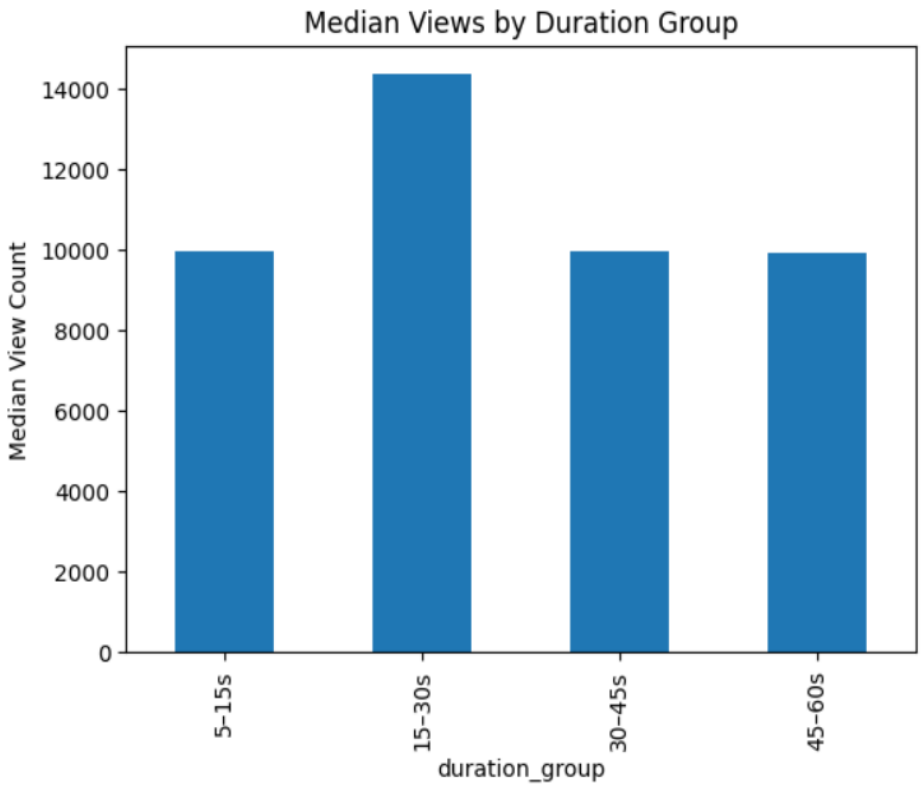
With all the durations split into groups, we will begin to group the view count and duration into its respective duration group. With a large sample like this, we will calculate the median for each group and compare its performance with the duration (which will be compared in section 4: Analysis).

```
views_by_duration = data.groupby("duration_group")["video_view_count"].median()
likes_by_duration = data.groupby("duration_group")["video_like_count"].median()
shares_by_duration = data.groupby("duration_group")["video_share_count"].median()
comments_by_duration = data.groupby("duration_group")["video_comment_count"].median()
```

4. Analysis

Multiple plots of median engagement metrics against different duration groups will be produced to investigate their trends and patterns.

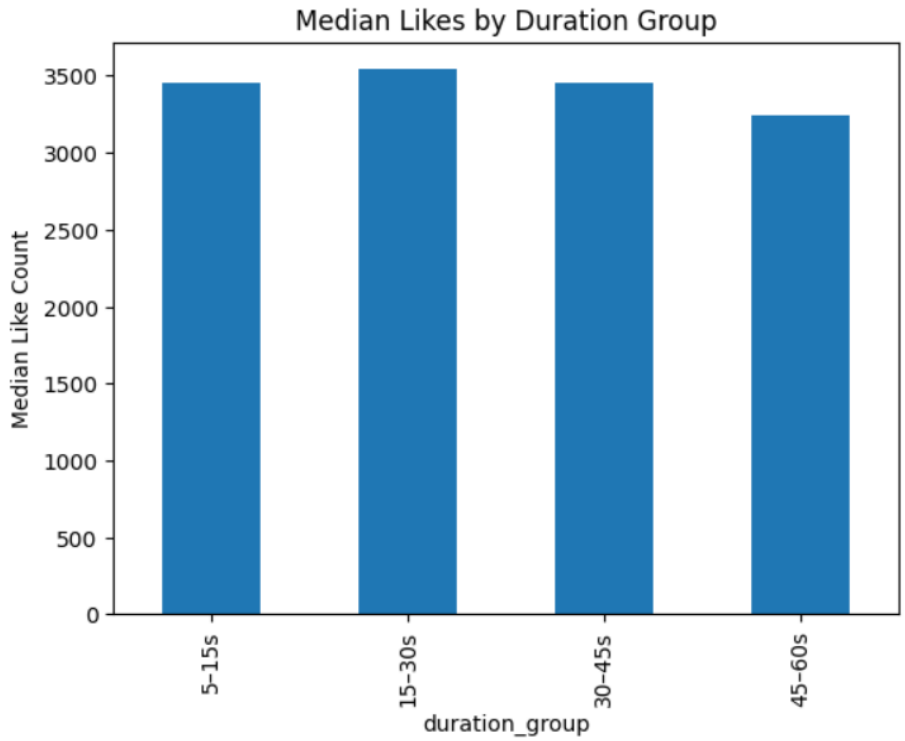
```
views_by_duration.plot(kind='bar')
plt.title("Median Views by Duration Group")
plt.ylabel("Median View Count")
```



- The 15-30 second category showed the highest median view count, suggesting that videos with this length tend to perform better than shorter or longer videos. The remaining duration groups all had similar median view counts, indicating that short videos do not really receive more views than longer ones.
- This also suggests that TikTok users may prefer videos that are long enough to tell a story but also short enough to maintain their attention span.
- The best duration to maximising views appears to be in the range of group 2: 15-30 seconds.

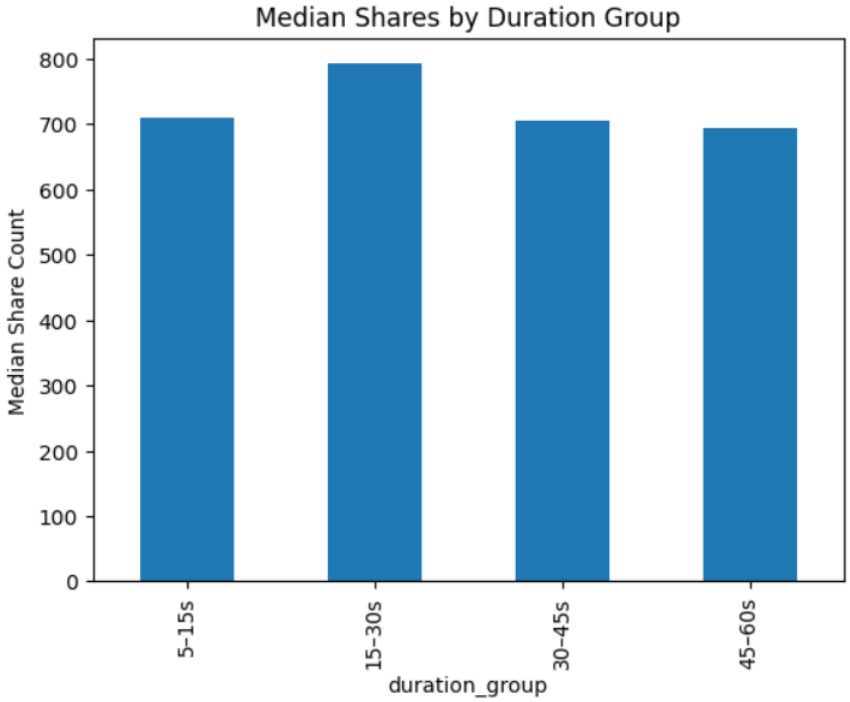
Very good. You can also go further by plotting the boxplot for each group to see the range and how it varies between groups.

```
likes_by_duration.plot(kind='bar')
plt.title("Median Likes by Duration Group")
plt.ylabel("Median Like Count")
```



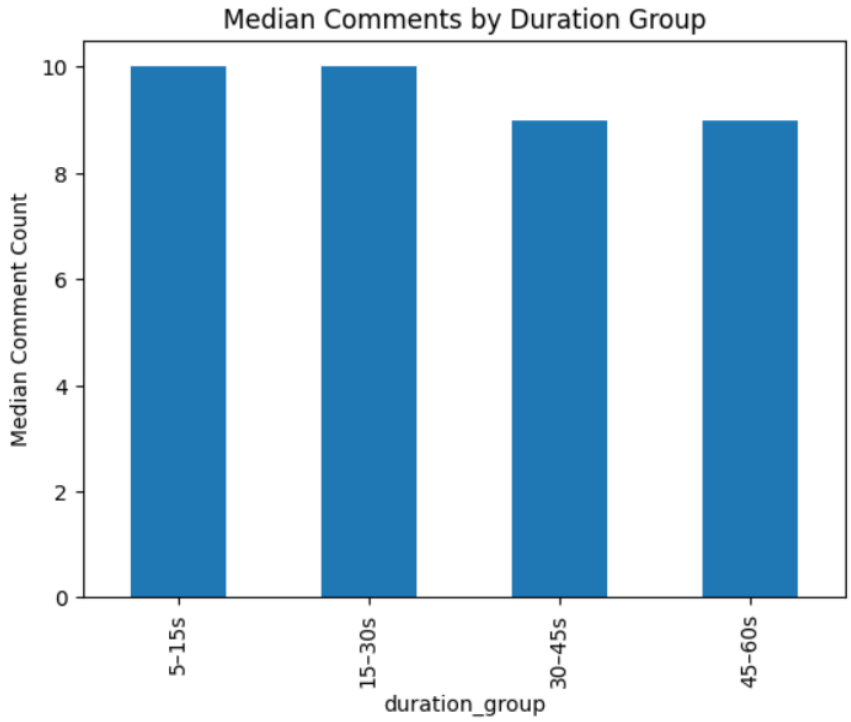
- The like count is more stable than the view count, the plot seems to be consistent across the first three duration groups.
- But the 15-30 second category again, appears to have a little bit higher median, while the longest duration group shows a noticeable drop.
- This suggests that video length does not strongly influence on the typical number of likes, unless if the video is in the longer duration range (45-60 seconds).

```
shares_by_duration.plot(kind='bar')
plt.title("Median Shares by Duration Group")
plt.ylabel("Median Share Count")
```



- The shares plot follows a similar pattern to views.
- Again, the 15-30 second group seems to perform better than the other groups, while the remaining show very similar share counts.
- The last group (45-60 seconds) has a slight fewer share than the others.
- This indicates that mid-length videos are more likely to be shared compared to both short and long videos.

```
comments_by_duration.plot(kind='bar')
plt.title("Median Comments by Duration Group")
plt.ylabel("Median Comment Count")
```



- Median comment counts are higher for the short duration groups (5-15s and 15-30s) and decreases a little for the other two groups.
- There is only a very small decline in comments for videos that are above 30 seconds.
- The differences may be small, but this pattern suggests that shorter videos attract slightly more interactions, while longer videos receive fewer responses.
- All videos, regardless of length, tend to receive around the same median number of comments (9-10).
- This indicates that comment engagement is only affected by video duration on a small scale.

Does video duration affect engagement?

Overall, video duration has a selective impact on user engagement. View and share counts display a pronounced peak within the 15-30 second range, suggesting that users are more likely to watch, rewatch, and share mid-length TikTok videos.

In contrast, likes stay consistent across most durations, and comments show little variation. These findings imply that duration influences passive engagement behaviors (viewing and sharing) more than active behaviors (liking and commenting). The duration itself does not entirely determine a video's performance, but it had shown that mid-length videos have advantages in reaching and engaging more audiences.

Very good observation!

5. Conclusion

To sum up, this analysis shows that video duration engagement does influence TikTok engagement. But it is important to note that its impact is different across different engagement metrics. Videos between 15 and 30 seconds frequently had the most median views and shares, indicating that people are more likely to watch and share content within this duration range. The analysis also suggests that the number of likes depends more on the quality of the content rather than the video length, as its median measurements remain consistent across all duration groups. Similarly, the length of the video does not significantly influence the comment count due to the small differences observed within these groups.

Another key insight from these results is that TikTok users and creators can aim for this range (15-30 seconds) to maximise their reaching potential to the audiences, even though as mentioned above, duration alone does not define if a video will get viral.

Based on all the findings above, one can conclude that audiences tend to prefer mid-length videos, particularly those from 15 to 30 seconds over very short or longer videos, suggesting a specific preference for this duration range. However, as the dataset only includes videos up to 60 seconds, we should only apply these conclusions on short-form content and not on longer formats.

Very good project.

- Interesting topic & data.
- Good use of Python for EDA
- Proper use of graphs and explanations

Consideration: the project is currently looking at high level / single attribute and its influence. Combining them may make the story more affirmative. Also try to think what other metrics you can derive from this dataset, for example: like / comment per view, ...